Github 账号： GlueNa2SiO3

个人博客关于密码学实验的链接:

https://github.com/Glue-Na2SiO3/GLUE

实验题目：密码学第三次实验

实验摘要:

本实验围绕 RSA 加密算法展开两项核心任务，旨在深入理解 RSA 的数学原理与工程实现逻辑。第一项任务聚焦 RSA 密钥选择的安全性，基于给定素数 p=1009 和 q=3643，寻找使未隐藏消息数最少的有效公钥指数 e 并计算其总和，验证了未隐藏消息数与数论中最大公约数的关联特性。第二项任务完成了 RSA 加密算法的完整工程实现，包括随机素数生成、密钥对创建、大整数与字节串转换、数据加解密等核心功能，通过测试验证了算法的正确性。实验结果表明，RSA 算法的安全性依赖于大素数分解的困难性，且密钥生成过程中的参数筛选（如 e 的选择、素数的有效性）直接影响算法的安全性与可用性。

题目描述

题目 1：未隐藏消息数最少的 e 值求解

(1)核心背景：RSA 加密中，若消息 m 满足 m^e ≡ m mod n（n=pq），则 m 为未隐藏消息，需使此类消息数最少。

(2)已知条件：给定素数 p=1009 和 q=3643，需计算欧拉函数 φ=(p-1)(q-1)。

(3)任务要求：寻找所有满足 1<e<φ且 gcd(e,φ)=1 的 e，要求其对应的未隐藏消息数最少，最终计算这些 e 的总和。

题目 2：RSA 加密算法完整实现

(1)核心要求：基于 RSA 数学原理，实现从密钥生成到数据加解密的全流程。

(2)具体任务：

生成指定比特长度的随机素数（通过小素数因子检测和素性测试保证有效性）；

固定公钥指数 e=3，计算私钥指数 d（e 模 φ的乘法逆元）；

实现字节串与大整数的双向转换，支持字符串数据的加解密；

通过测试用例（如字符串"test"、数字 42）验证加解密一致性。

过程

1. 未隐藏消息数最少的 e 值求解

(1)数学原理与核心逻辑

未隐藏消息数的计算依赖中国剩余定理和循环群性质：

若 m^e ≡ m mod n（n=pq），则等价于 m^e ≡ m mod p 和 m^e ≡ m mod q 同时成立；

对素数 p，方程 m^e ≡ m mod p 的解数为 1 + gcd(e-1, p-1)（1 对应 m=0，gcd(e-1, p-1) 对应非零解）；

未隐藏消息总数为 p 和 q 对应解数的乘积，需筛选使两者均为最小值的 e。

2. 核心实现步骤与代码

（1）关键函数实现（未隐藏消息数计算）

```
import math

def count_unhidden(modulus, exponent):
    """计算素数模下满足 m^e ≡ m mod modulus 的解数"""
    count = 0
    for m in range(modulus):
        if pow(m, exponent, modulus) == m:
            count += 1
    return count

def get_unhidden_counts(prime):
    """获取素数 prime 对应的所有有效 e 的未隐藏消息数列表"""
    mod = prime - 1
    counts = []
    for e in range(mod):
        if math.gcd(e, mod) != 1:
            counts.append(float('inf'))  # 非互素 e 标记为无效
        else:
            counts.append(count_unhidden(prime, e))
    return counts
```

（2）e 值筛选与求和核心逻辑

```
def calculate():
    p = 1009
    q = 3643
    phi = (p - 1) * (q - 1)  # 计算欧拉函数 φ

    # 计算 p 和 q 对应的最小未隐藏消息数
    p_counts = get_unhidden_counts(p)
    min_p = min(p_counts)
    q_counts = get_unhidden_counts(q)
    min_q = min(q_counts)

    # 筛选满足条件的 e 并求和（1<e<phi，gcd(e,phi)=1，解数均为最小值）
    total = 0
    for e in range(2, phi):
        if math.gcd(e, phi) != 1:
            continue
        # 利用模的性质简化判断（e mod (p-1)对应 p 的解数，e mod (q-1)对应 q 的解数）
        if p_counts[e % (p - 1)] == min_p and q_counts[e % (q - 1)] == min_q:
            total += e
```

return str(total)

3. 执行结果



1. RSA 加密算法完整实现

(1)数学原理与核心逻辑

RSA 实现需覆盖 “素数生成→密钥创建→数据转换→加解密” 四大核心环节：

素数生成：先排除小素数因子，再通过素性测试保证有效性；

密钥创建：固定 e=3，确保(p-1)和(q-1)不被 3 整除（保证 gcd(e, ϕ)=1），用扩展欧几里得算法求 d；

数据转换：实现字节串与大整数的双向转换，支持字符串加密；

加解密：基于模幂运算实现核心数学逻辑，保证效率与正确性。

2. 核心实现步骤与代码

（1）基础工具函数（模逆、素性测试）

```
import random
import binascii
```

```
def mod_inverse(a, n):
    """基于扩展欧几里得算法求解 a 模 n 的乘法逆元"""
    temp_t, new_temp = 0, 1
    remainder, new_remainder = n, a
    while new_remainder != 0:
        quotient = remainder // new_remainder
        remainder, new_remainder = new_remainder, remainder - quotient *
new_remainder
        temp_t, new_temp = new_temp, temp_t - quotient * new_temp
    if remainder > 1:
        raise ValueError("No modular inverse exists")
    return temp_t + n if temp_t < 0 else temp_t

def is_likely_prime(p, rounds=5):
    """费马素性测试：判断 p 是否为大概率素数"""
    if p % 2 == 0:
        return p == 2  # 快速排除偶数（除 2 外）
    for _ in range(rounds):
        a = random.randint(1, p - 1)
        if pow(a, p - 1, p) != 1:
            return False
    return True
```

（2）素数生成与密钥创建

```
def get_likely_prime(bit_length):
    """生成指定比特长度的大概率素数"""
    small_prime_factors = [2, 3, 5, 7, 11, 13, 17, 19, 23]
    def has_small_factor(num):
        """判断是否含小素数因子"""
        for prime in small_prime_factors:
            if num % prime == 0:
                return True
        return False

    while True:
        p = random.getrandbits(bit_length)
        p |= (1 << (bit_length - 1)) | 1  # 保证最高位为 1（满足比特长度）且为奇数
        if not has_small_factor(p) and is_likely_prime(p):
            return p

def generate_rsa_keys(key_size):
    """生成 RSA 密钥对（公钥：(e,n)，私钥：(d,n)）"""
    public_exp = 3
    half_bits = (key_size + 1) // 2 + 1  # 每个素数的比特数（保证 n 达到指定密钥长度）

    # 生成 p：确保(p-1)不被 3 整除（保证 gcd(e, φ)=1）
    p = 7
    while (p - 1) % public_exp == 0:
        p = get_likely_prime(half_bits)
```

```
    # 生成 q：确保 q≠p 且(q-1)不被 3 整除
    q = p
    while q == p or (q - 1) % public_exp == 0:
        q = get_likely_prime(half_bits)

    modulus = p * q
    totient = (p - 1) * (q - 1)
    private_exp = mod_inverse(public_exp, totient)
    return (public_exp, modulus), (private_exp, modulus)
```

（3）数据转换与加解密核心函数

```
def bytes_to_num(byte_str):
    """字节串转大整数"""
    return int(binascii.hexlify(byte_str), 16) if byte_str else 0

def num_to_bytes(num):
    """大整数转字节串（无前置零）"""
    if num == 0:
        return b""
    byte_length = (num.bit_length() + 7) // 8
    return num.to_bytes(byte_length, byteorder='big')

def encrypt_number(pub_key, m):
    """整数加密：c = m^e mod n"""
    e, n = pub_key
    if not (0 <= m < n):
        raise ValueError(f"Message {m} out of range [0, {n-1}]")
    return pow(m, e, n)

def decrypt_number(priv_key, c):
    """整数解密：m = c^d mod n"""
    d, n = priv_key
    if not (0 <= c < n):
        raise ValueError(f"Ciphertext {c} out of range [0, {n-1}]")
    return pow(c, d, n)

def encrypt_byte_string(pub_key, byte_data):
    """字节串加密（支持字符串转换）"""
    m = bytes_to_num(byte_data)
    c = encrypt_number(pub_key, m)
    return num_to_bytes(c)

def decrypt_byte_string(priv_key, encrypted_data):
    """字节串解密"""
    c = bytes_to_num(encrypted_data)
    m = decrypt_number(priv_key, c)

    return num_to_bytes(m)
```

3. 执行结果,测试验证



```
112
113 if __name__ == '__main__':
114     # 测试128位密钥
115     pub, priv = generate_rsa_keys(128)
116     test_data = b'test RSA implementation'
117     encrypted = encrypt_byte_string(pub, test_data)
118     decrypted = decrypt_byte_string(priv, encrypted)
119
120     if test_data != decrypted:
121         raise AssertionError(f"Test failed: original {test_data}, decrypted {decrypted}")
122     print(f"测试成功！原始数据：{test_data.decode()}，解密后：{decrypted.decode()}")
123
124     # 测试数字加密
125     test_num = 42
126     encrypted_num = encrypt_number(pub, test_num)
127     decrypted_num = decrypt_number(priv, encrypted_num)
128     assert test_num == decrypted_num, f"数字测试失败：原始{test_num}，解密后{decrypted_num}"
129     print(f"数字测试成功：原始{test_num}，解密后{decrypted_num}")
```

问题 4 输出 调试控制台 终端 端口 SPELL CHECKER 4

```
测试成功！原始数据：test RSA implementation，解密后：test RSA implementation
数字测试成功：原始42，解密后42
```

总结

（一）实验收获

理论层面：深入理解了 RSA 算法的数学基础，包括欧拉函数、模逆运算、中国剩余定理的应用，明确了未隐藏消息数与 gcd(e-1, p-1)、gcd(e-1, q-1)的关联，掌握了素数生成与素性测试的核心逻辑。

实践层面：完成了从数学原理到工程实现的转化，掌握了大整数处理、字节串与整数转换等关键技术，解决了 RSA 实现中 “素数有效性筛选” “模逆存在性保证” 等实际问题。

安全性认知：明确了 e 的选择对 RSA 安全性的影响（如 e=3 时需确保(p-1)、(q-1)不被 3 整除），理解了未隐藏消息数最少的设计意义（减少信息泄露风险）。

（二）存在不足与优化方向

素性测试：当前使用费马素性测试，存在一定误判概率，可引入米勒 - 拉宾素性测试提升素数验证的准确性；

密钥长度：实验使用 128 位密钥，仅适用于测试，实际应用中需使用 2048 位及以上密钥，增强抗大素数分解的能力；

数据处理：未处理长字符串分片加密问题，可扩展实现对超长数据的分段加解密功能；

代码优化：部分函数逻辑可进一步精简（如素数生成中的随机数生成方式），提升运行效率。

（三）实验结论

本实验通过两项任务全面验证了 RSA 加密算法的原理与可行性。第一项任务明确了密钥参数选择与消息安全性的关联，第二项任务实现了完整的 RSA 加解密流程并通过测试验证了正确性。实验表明，RSA 算法的核心在于数学原理的严谨性与工程实现的细节把控，合理的参数选择与规范的实现流程是保证算法安全性与可用性的关键。

参考文献

无